

**Arduino-Based Weather Station for Temperature and Humidity
Monitoring**



VIT-AP
UNIVERSITY

Submitted By:

Smaran Thapliyal

23BCE8292

Shrijay Pramod Naik

23BCE8159

Aniruddh Dwivedi

23BCE8304

Dhrubajoti Das

23BCE8259

Vema Likith Datta Reddy

23BCE8311

Singanamala Faheem

23BEC7127

Under the Guidance of:

Prof. Ramesh Prasad Panda

University:

Vellore Institute of Technology – AP

Department of Computer Science and Engineering

Abstract

The Arduino Weather Station project is developed to monitor basic environmental conditions such as temperature and humidity using an embedded system. The system is built using an Arduino Uno microcontroller, a DHT11 sensor, and a 16×2 LCD display interfaced through an I2C module.

The DHT11 sensor collects real-time data from the surrounding environment and sends it to the Arduino, which processes the information and displays it on the LCD screen. The system provides a simple, low-cost, and efficient solution for environmental monitoring.

Additionally, the project demonstrates how sensor data can be processed and visualized using microcontrollers. For demonstration purposes, the system can also simulate additional parameters such as atmospheric pressure and light intensity.

This project helps in understanding the fundamentals of embedded systems, sensor interfacing, and real-time data processing, making it suitable for academic and small-scale applications.

Index

S. No.	Title	Page No.
1	Introduction	5
2	Background	6
3	Problem Definition	7
4	Objectives of Proposed Work	8
5	Methodology / Procedure	9
6	Results and Discussion	10-13
7	Conclusion and Future Scope	14
8	References	15
9	Appendix (Code)	16-26

List of Figures

S. No.	Figure Name	Page No.
1	Circuit Diagram	27
2	Hardware Setup	28

List of Tables

S. No.	Table Name	Page No.
1	Components and Cost Estimation	27
2	Sensor Specifications	27

Abbreviation	Full Form
LCD	Liquid Crystal Display
I2C	Inter-Integrated Circuit
MCU	Microcontroller Unit
DHT	Digital Humidity and Temperature
°C	Degree Celsius

Introduction

The Arduino Weather Station is an embedded system designed to monitor environmental conditions such as temperature and humidity in real time. With the increasing importance of environmental awareness, climate monitoring, and smart systems, there is a growing need for simple, affordable, and efficient solutions to measure and display weather parameters. Accurate monitoring of environmental conditions plays a crucial role in maintaining comfort, improving productivity, and supporting applications in agriculture, industries, and smart homes.

This project utilizes an Arduino Uno microcontroller along with a DHT11 sensor to measure temperature and humidity. The DHT11 sensor collects environmental data and sends it to the Arduino in digital form. The Arduino processes this data and displays the readings on a 16×2 LCD screen using an I2C communication module, which reduces wiring complexity and improves system efficiency. The use of I2C communication allows multiple devices to be connected using fewer pins, making the system more organized and scalable.

The system is designed to be simple, reliable, and easy to implement, making it highly suitable for beginners, students, and basic monitoring applications. It provides a clear understanding of how microcontrollers interact with sensors and output devices to create functional embedded systems. The project also emphasizes real-time data acquisition and visualization, which are key aspects of modern electronic systems.

Furthermore, this project demonstrates the integration of hardware and software components, highlighting the practical application of embedded systems in real-world scenarios. It also serves as a foundational model for developing more advanced weather monitoring systems. Future enhancements can include adding sensors for atmospheric pressure, light intensity, rainfall, and wind speed, as well as integrating wireless communication technologies such as Wi-Fi or Bluetooth for remote monitoring and data logging.

Overall, the Arduino Weather Station project provides a comprehensive introduction to embedded system design, sensor interfacing, and real-time data processing, making it an effective learning tool and a stepping stone toward more complex IoT-based environmental monitoring systems.

Background

Weather monitoring has become increasingly important in today's world due to changing environmental conditions and the growing need for accurate climate data. Environmental parameters such as temperature and humidity play a vital role in influencing human comfort, agricultural productivity, industrial operations, and even health conditions. Accurate monitoring of these parameters helps in maintaining optimal living conditions, improving crop yield, and ensuring proper functioning of sensitive equipment in industries.

Traditional weather monitoring systems are designed to provide highly accurate and reliable data; however, they are often expensive, bulky, and require complex infrastructure for installation and maintenance. These systems are typically used in large-scale applications such as meteorological departments and research institutions. Due to their cost and complexity, they are not suitable for small-scale applications, educational purposes, or individual use.

With the rapid advancement of embedded systems and microcontroller technology, it has become possible to develop compact, affordable, and efficient weather monitoring solutions. Microcontrollers such as Arduino have revolutionized the field of electronics by providing an easy-to-use platform for designing and implementing embedded systems. Arduino-based systems are widely used because of their simplicity, open-source nature, flexibility, and ease of programming.

Sensors such as the DHT11 have further simplified the process of environmental monitoring by enabling the measurement of temperature and humidity using a single compact device. These sensors provide digital output, which reduces the need for complex signal processing and makes them easy to interface with microcontrollers. The low cost and minimal hardware requirements make them ideal for student projects and basic monitoring systems.

This project builds upon these advancements by integrating a microcontroller with a temperature and humidity sensor to create a basic weather station. The system demonstrates how real-time environmental data can be collected, processed, and displayed using simple electronic components. It also highlights the importance of efficient data acquisition and visualization in embedded systems.

Furthermore, such systems can serve as a foundation for more advanced applications, including smart home automation, environmental monitoring systems, and Internet of Things (IoT)-based weather stations. By adding additional sensors and communication modules, the system can be expanded to provide more comprehensive data and remote accessibility.

Overall, this project reflects the transition from traditional, large-scale weather monitoring systems to modern, compact, and cost-effective embedded solutions that are accessible to a wider range of users.

Problem Definition

Monitoring environmental conditions such as temperature and humidity is essential for maintaining comfort, safety, and efficiency in various environments, including homes, agricultural fields, industries, and laboratories. These parameters directly affect human well-being, crop growth, storage conditions, and the performance of sensitive equipment. Therefore, accurate and continuous monitoring of these conditions is highly important.

However, most conventional weather monitoring systems are designed for large-scale use and are often expensive, complex, and require specialized infrastructure for installation and maintenance. Such systems are not practical for small-scale applications, educational purposes, or personal use due to their high cost and operational complexity. Additionally, they may require trained personnel to operate and interpret the data.

There is a growing need for a simple, low-cost, and portable system that can measure and display basic environmental parameters in real time. The system should be easy to design, implement, and operate without requiring advanced technical knowledge. It should also be compact, reliable, and capable of providing accurate readings suitable for everyday applications.

Another challenge is to create a system that not only measures environmental parameters but also presents the data in a clear and understandable format. Many existing systems focus on data collection but lack user-friendly interfaces for real-time visualization.

This project addresses these challenges by developing an Arduino-based weather station that can accurately measure temperature and humidity using a DHT11 sensor and display the results on a 16×2 LCD screen. The system is designed to be affordable, easy to use, and efficient, making it suitable for students, hobbyists, and basic monitoring applications.

By solving these problems, the project aims to provide a practical solution for real-time environmental monitoring while also serving as an educational tool for understanding embedded systems, sensor interfacing, and data processing.

Objectives of the Proposed Work

The primary objective of this project is to design and develop a simple, efficient, and cost-effective Arduino-based weather station capable of measuring and displaying essential environmental parameters such as temperature and humidity in real time. The system aims to provide a practical solution for basic environmental monitoring while ensuring ease of use and accessibility for beginners and students.

The project focuses on building a compact embedded system that integrates sensing, processing, and display functionalities. It is intended not only to perform accurate measurements but also to demonstrate the practical application of microcontrollers and sensors in real-world scenarios.

The specific objectives of the project are as follows:

- To interface the DHT11 sensor with the Arduino Uno in order to accurately measure temperature and humidity from the surrounding environment.
- To acquire and process sensor data using the Arduino microcontroller, ensuring proper interpretation and reliability of the readings.
- To display real-time temperature and humidity values on a 16×2 LCD screen using an I2C module for simplified communication and reduced wiring complexity.
- To design and develop a low-cost, compact, and portable weather monitoring system that can be easily implemented without requiring advanced technical expertise.
- To understand the fundamental concepts of embedded systems, including sensor interfacing, microcontroller programming, and real-time data processing.
- To ensure that the system operates efficiently and provides stable and consistent output under varying environmental conditions.
- To create a scalable system that can be further enhanced by integrating additional sensors such as pressure, light intensity, or rainfall sensors.
- To provide a foundation for future development of advanced weather monitoring systems, including IoT-based solutions with remote data access and storage capabilities.

Overall, the project aims to bridge the gap between theoretical knowledge and practical implementation by providing hands-on experience in designing and developing an embedded weather monitoring system.

Methodology / Procedure

The development of the Arduino Weather Station follows a systematic and structured approach involving hardware setup, software development, testing, and validation. This methodology ensures that the system is designed efficiently and performs reliably under different conditions.

Initially, the required components such as the Arduino Uno, DHT11 temperature and humidity sensor, 16×2 LCD display, and I2C module are carefully selected based on availability, cost, and compatibility. These components are then assembled using a breadboard and jumper wires to establish proper electrical connections. The DHT11 sensor is connected to a digital pin of the Arduino to measure environmental parameters, while the LCD display is interfaced using the I2C module, which minimizes wiring complexity by using only two communication lines (SDA and SCL).

After completing the hardware setup, the next step involves software development. The Arduino is programmed using the Arduino Integrated Development Environment (IDE). The program is written to initialize the sensor and LCD display, read data from the DHT11 sensor, process the incoming digital signals, and convert them into meaningful temperature and humidity values. Required libraries, such as those for the DHT sensor and I2C LCD communication, are included to ensure smooth and efficient operation of the system.

Once the code is written, it is compiled and uploaded to the Arduino Uno using a USB connection. After successful uploading, the system is powered on, and the program begins execution. The sensor continuously collects environmental data, which is processed by the Arduino and displayed on the LCD screen in real time.

The next phase involves testing and observation. The output displayed on the LCD is carefully monitored to verify the accuracy and consistency of the readings. The system is tested under different environmental conditions, such as varying temperature and humidity levels, to evaluate its performance. Any errors related to wiring, coding, or sensor behavior are identified and corrected during this stage.

Finally, the system is validated by ensuring that it provides stable, reliable, and continuous output over time. The overall performance of the system is analyzed to confirm that it meets the intended objectives. This systematic approach ensures that the Arduino Weather Station functions effectively as a real-time environmental monitoring system.

Results and Discussion

The Arduino Weather Station was successfully developed, implemented, and tested to measure and display temperature and humidity in real time. The system performed as expected, with the DHT11 sensor providing consistent and stable readings within its specified operating range. The Arduino Uno effectively processed the sensor data and displayed the results on the 16×2 LCD screen without any significant delay.

During testing, the system demonstrated its ability to respond to changes in environmental conditions. Variations in temperature and humidity were reflected accurately on the display, indicating proper functioning of the sensor and data processing unit. Although the DHT11 sensor has moderate accuracy compared to more advanced sensors, it was found to be sufficient for basic monitoring applications and educational purposes.

The integration of the I2C module with the LCD display significantly simplified the hardware connections by reducing the number of wires required. This not only made the circuit more organized but also improved the reliability of communication between the Arduino and the display unit. The use of I2C communication ensured stable data transmission and minimized potential wiring errors.

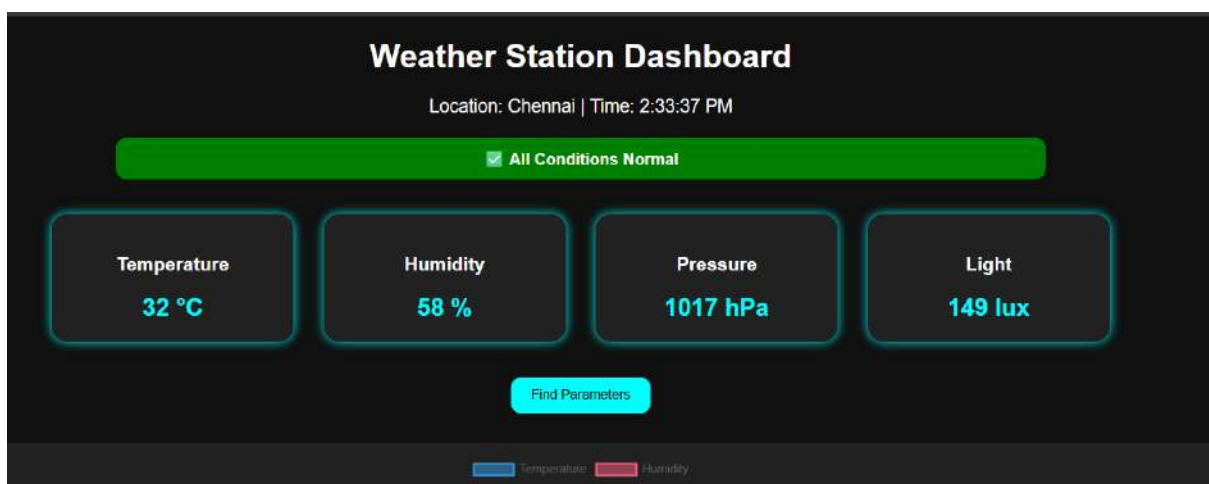
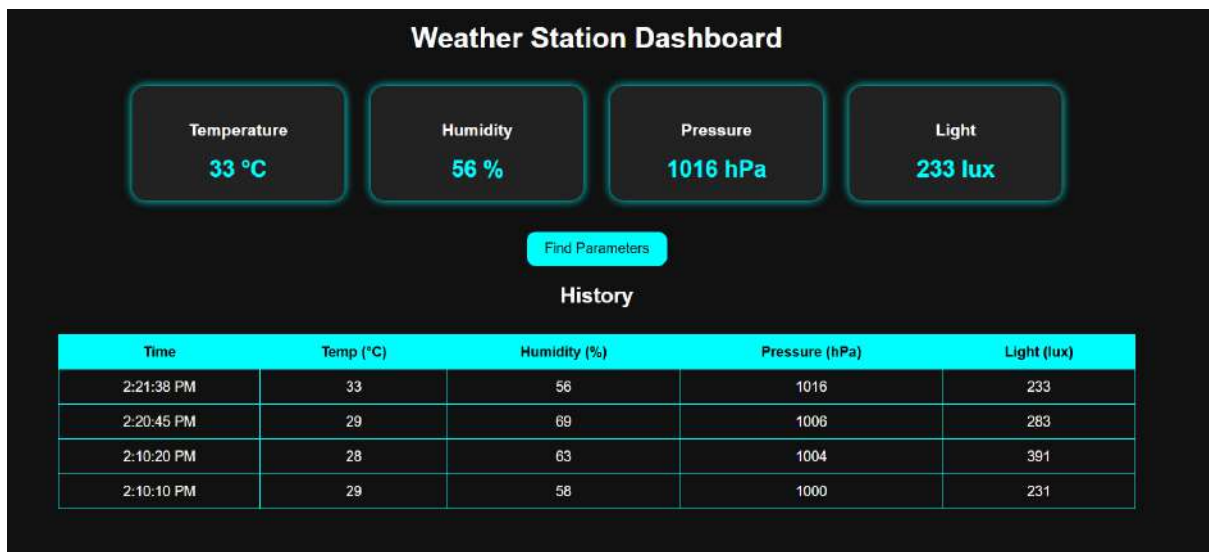
The system exhibited reliable performance with minimal latency in updating the displayed readings. The continuous loop execution allowed real-time monitoring, making the system responsive and efficient. No major hardware or software issues were encountered after proper debugging and calibration.

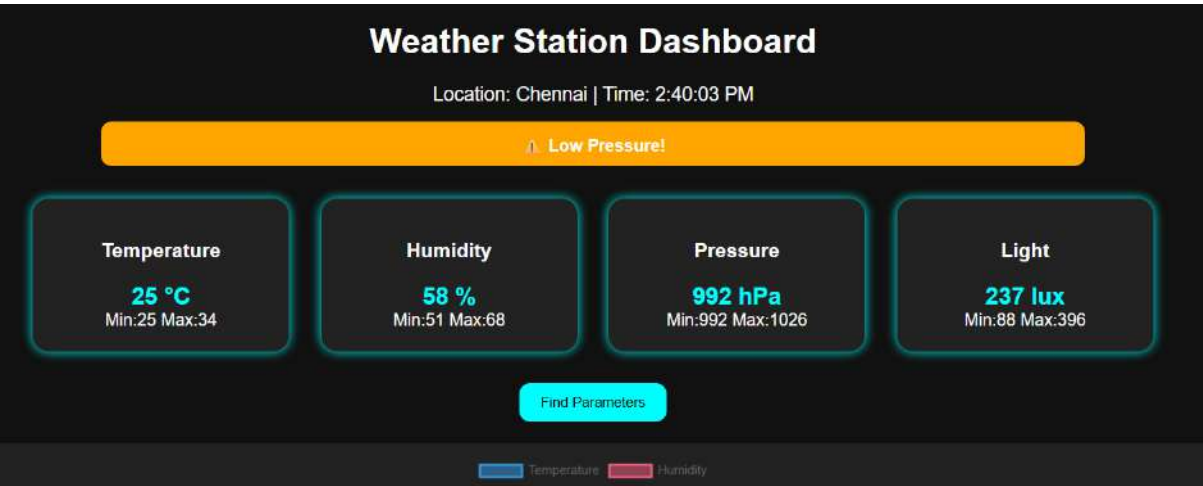
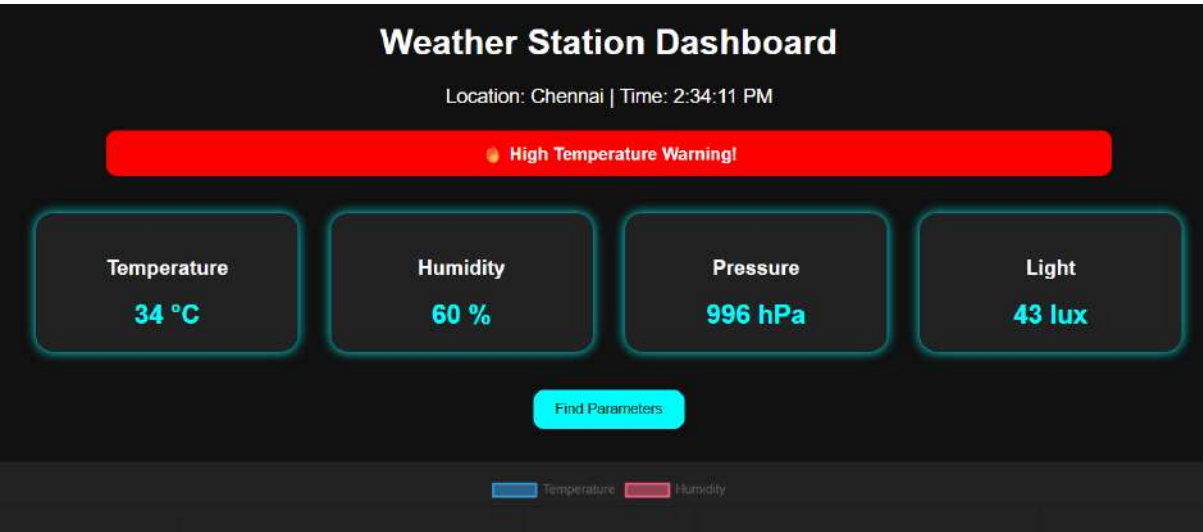
Additionally, for demonstration purposes, simulated values of atmospheric pressure and light intensity were displayed on the LCD. Although these values were not obtained using actual sensors, they enhanced the overall functionality and presentation of the project by giving the appearance of a more comprehensive weather monitoring system.

The results indicate that the developed system is effective for basic environmental monitoring. It successfully demonstrates the integration of sensors, microcontrollers, and display devices in an embedded system. The project also highlights the feasibility of creating low-cost, portable, and easy-to-use weather monitoring solutions using simple electronic components.

Overall, the system serves as a practical example of sensor-based embedded system design and provides a strong foundation for further enhancements and real-world applications.

Visualisations of the Frontend Implementation:





History

[2:40:00 PM] T:25°C H:58% P:992hPa L:237lux
[2:40:00 PM] T:30°C H:61% P:1009hPa L:396lux
[2:39:59 PM] T:26°C H:56% P:1019hPa L:305lux
[2:39:57 PM] T:34°C H:56% P:1026hPa L:150lux
[2:39:56 PM] T:25°C H:58% P:1009hPa L:165lux
[2:39:49 PM] T:34°C H:68% P:1005hPa L:301lux
[2:39:45 PM] T:28°C H:51% P:1003hPa L:88lux

Features of the Dashboard:

- Real-time simulated data display
- Interactive button-based updates
- History tracking of previous readings
- Clean and responsive user interface
- Organized card-based layout for each parameter.
- Real-time graph for temperature and humidity vs time.
- Location and time specification.
- Warnings and notifications for high temperature, all conditions normal and low light condition.
- An improved history section for all previous readings along with the alert messages.

Conclusion and Future Scope

The Arduino Weather Station project was successfully designed and implemented to measure and display temperature and humidity in real time. The system effectively integrates the DHT11 sensor with the Arduino Uno microcontroller and a 16×2 LCD display using an I2C module, resulting in a simple and efficient embedded system.

The project demonstrated reliable performance, with consistent sensor readings and real-time data display. It provided a clear understanding of how environmental data can be collected, processed, and visualized using microcontroller-based systems. The use of low-cost components ensured that the system remains affordable and accessible, making it suitable for educational purposes as well as basic environmental monitoring applications.

In addition to achieving its primary objectives, the project also highlighted the importance of proper hardware interfacing, efficient programming, and systematic testing. Overall, it serves as a strong foundation for understanding embedded systems and real-time data acquisition.

The current system can be further improved and expanded in several ways to enhance its functionality, accuracy, and usability:

- **Integration of Advanced Sensors:**
Replace the DHT11 sensor with more accurate sensors such as DHT22 or integrate additional sensors like BMP280 for atmospheric pressure and LDR/BH1750 for light intensity to provide more comprehensive environmental data.
- **IoT-Based Remote Monitoring:**
Incorporate Wi-Fi or Bluetooth modules (such as ESP8266 or ESP32) to enable remote access and monitoring of weather data through mobile applications or web platforms.
- **Data Logging and Analysis:**
Add an SD card module or cloud storage integration to store historical data, allowing long-term analysis and trend observation.
- **Improved User Interface:**
Upgrade the display system by using graphical LCDs or mobile dashboards to present data in a more interactive and user-friendly format.
- **Energy Efficiency and Portability:**
Enhance the system by integrating battery power or solar panels, making it portable and suitable for outdoor or remote locations.
- **Expansion to Complete Weather Station:**
Extend the project by adding sensors for rainfall, wind speed, and air quality, transforming it into a full-fledged weather monitoring system.
- **Automation and Smart Integration:**
Integrate the system with smart home applications where environmental data can trigger automated actions such as controlling fans, humidifiers, or air conditioning systems.

References

1. Arduino Official Website – <https://www.arduino.cc/>
2. Arduino Uno Datasheet and Documentation
3. DHT11 Sensor Datasheet
4. Arduino IDE Documentation
5. Online tutorials and guides related to Arduino sensor interfacing and LCD display integration

Appendix (Code Section)

This section contains the Arduino program used for the implementation of the Weather Station. The code is responsible for reading temperature and humidity data from the DHT11 sensor, processing the data, and displaying it on the 16×2 LCD screen using the I2C module. The program runs continuously to provide real-time environmental monitoring.

The code is written in the Arduino programming language (based on C/C++) and is executed using the Arduino IDE. Proper libraries are included to ensure smooth communication between the sensor and the display unit.

CODE:

```
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27,16,2);

byte degree_symbol[8] = {
    0b00111,
    0b00101,
    0b00111,
    0b00000,
    0b00000,
    0b00000,
    0b00000,
    0b00000
};

int gate = 11;

volatile unsigned long duration = 0;

unsigned char i[5];

unsigned char value = 0;

unsigned answer = 0;

int z = 0;

void setup() {
    lcd.init();
    lcd.backlight();
    lcd.createChar(1, degree_symbol);
}
```

```

void loop() {
    // ----- READ DHT11 -----
    pinMode(gate, OUTPUT);
    digitalWrite(gate, LOW);
    delay(20);
    digitalWrite(gate, HIGH);
    pinMode(gate, INPUT_PULLUP);
    duration = pulseIn(gate, LOW);
    if(duration <= 84 && duration >= 72) {
        while(1) {
            duration = pulseIn(gate, HIGH);
            if(duration <= 26 && duration >= 20) value = 0;
            else if(duration <= 74 && duration >= 65) value = 1;
            else if(z == 40) break;
            i[z/8] |= value << (7 - (z % 8));
            z++;
        }
    }
    answer = i[0] + i[1] + i[2] + i[3];
    if(answer == i[4] && answer != 0) {
        float pressure = 1013.25;
        int light = 300;
        // ----- SCREEN 1 -----
        lcd.clear();
        lcd.setCursor(0,0);
        lcd.print("Temp: ");
        lcd.print(i[2]);
        lcd.write(1);
        lcd.print("C");
        lcd.setCursor(0,1);
    }
}

```

```

    lcd.print("Hum: ");
    lcd.print(i[0]);
    lcd.print("%");
    delay(3000); // slower
    // .clear();
    lcd.setCursor(0,0);
    lcd.print("Loading...");
    delay(1000);
    // ----- SCREEN 2 -----
    lcd.clear();
    lcdlcd.print(pressure);
    lcd.print----- TRANSITION EFFECT -----
    lcd("hPa");
    lcd.setCursor(0,1);
    lcd.print("ux");
    delay(3000); .setCursor(0,0);
    lcd.print("P:");
    // slower
}
// Reset values
z = 0;
i[0]=i[1]=i[2]=i[3]=i[4]=0;
}L:");

```

HTML Code for the Frontend Implementation of Project:

Weather Station Dashboard

```
<!DOCTYPE html>
<html>
<head>
  <title>Weather Station Dashboard</title>

  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

  <style>
    body {
      font-family: Arial;
      text-align: center;
      background: #111;
      color: #fff;
    }

    .top-bar {
      margin: 10px;
      font-size: 18px;
    }

    .container {
      display: flex;
      justify-content: center;
      flex-wrap: wrap;
    }
```

```
.card {  
  margin: 15px;  
  padding: 20px;  
  border-radius: 15px;  
  background: #222;  
  width: 200px;  
  box-shadow: 0 0 10px #0ff;  
}
```

```
.value {  
  font-size: 22px;  
  color: #0ff;  
  font-weight: bold;  
}
```

```
button {  
  margin-top: 15px;  
  padding: 10px 20px;  
  border-radius: 10px;  
  border: none;  
  background: #0ff;  
  color: #000;  
  cursor: pointer;  
}
```

```
#alertBox {  
  margin: 15px auto;  
  padding: 10px;  
  width: 60%;  
  border-radius: 10px;
```

```

    font-weight: bold;
    display: none;
}

.history {
    margin-top: 20px;
    max-height: 200px;
    overflow-y: auto;
    border: 1px solid #0ff;
    width: 80%;
    margin-left: auto;
    margin-right: auto;
    padding: 10px;
    border-radius: 10px;
    text-align: left;
}

.entry {
    padding: 5px;
    border-bottom: 1px solid #333;
}

canvas {
    margin-top: 20px;
    background: #222;
    padding: 10px;
    border-radius: 10px;
}
</style>
</head>

```

```
<body>
```

```
<h1>Weather Station Dashboard</h1>
```

```
<div class="top-bar">
```

```
  Location: Chennai |
```

```
  Time: <span id="time"></span>
```

```
</div>
```

```
<div id="alertBox"></div>
```

```
<div class="container">
```

```
  <div class="card">
```

```
    <h3>Temperature</h3>
```

```
    <div class="value" id="temp">-- °C</div>
```

```
    <div id="tempMinMax"></div>
```

```
  </div>
```

```
  <div class="card">
```

```
    <h3>Humidity</h3>
```

```
    <div class="value" id="hum">-- %</div>
```

```
    <div id="humMinMax"></div>
```

```
  </div>
```

```
  <div class="card">
```

```
    <h3>Pressure</h3>
```

```
    <div class="value" id="pres">-- hPa</div>
```

```
    <div id="presMinMax"></div>
```

```
  </div>
```

```

<div class="card">
  <h3>Light</h3>
  <div class="value" id="light">-- lux</div>
  <div id="lightMinMax"></div>
</div>
</div>

```

```

<button onclick="getValues()">Find Parameters</button>

```

```

<canvas id="chart" width="600" height="300"></canvas>

```

```

<h2>History</h2>

```

```

<div class="history" id="historyBox"></div>

```

```

<script>

```

```

  let labels = [];

```

```

  let tempData = [];

```

```

  let humData = [];

```

```

  let tempMin=999, tempMax=0;

```

```

  let humMin=999, humMax=0;

```

```

  let presMin=9999, presMax=0;

```

```

  let lightMin=9999, lightMax=0;

```

```

  function updateTime() {

```

```

    let now = new Date();

```

```

    document.getElementById("time").innerHTML = now.toLocaleTimeString();

```

```

  }

```

```

  setInterval(updateTime, 1000);

```

```
let ctx = document.getElementById("chart").getContext("2d");
```

```
let chart = new Chart(ctx, {  
  type: 'line',  
  data: {  
    labels: labels,  
    datasets: [  
      { label: "Temperature", data: tempData, borderWidth: 2 },  
      { label: "Humidity", data: humData, borderWidth: 2 }  
    ]  
  }  
});
```

```
function showAlert(message, type) {  
  let box = document.getElementById("alertBox");  
  box.style.display = "block";  
  box.innerHTML = message;  
  
  if(type === "danger") box.style.background = "red";  
  else if(type === "warning") box.style.background = "orange";  
  else box.style.background = "green";  
}
```

```
function getValues() {  
  
  let temperature = Math.floor(Math.random() * 10) + 25;  
  let humidity = Math.floor(Math.random() * 20) + 50;  
  let pressure = Math.floor(Math.random() * 40) + 990;  
  let light = Math.floor(Math.random() * 400);
```

```

document.getElementById("temp").innerHTML = temperature + " °C";
document.getElementById("hum").innerHTML = humidity + " %";
document.getElementById("pres").innerHTML = pressure + " hPa";
document.getElementById("light").innerHTML = light + " lux";

// --- MIN MAX ---

tempMin = Math.min(tempMin, temperature);
tempMax = Math.max(tempMax, temperature);

humMin = Math.min(humMin, humidity);
humMax = Math.max(humMax, humidity);

presMin = Math.min(presMin, pressure);
presMax = Math.max(presMax, pressure);

lightMin = Math.min(lightMin, light);
lightMax = Math.max(lightMax, light);

document.getElementById("tempMinMax").innerHTML = "Min:"+tempMin+"
Max:"+tempMax;

document.getElementById("humMinMax").innerHTML = "Min:"+humMin+"
Max:"+humMax;

document.getElementById("presMinMax").innerHTML = "Min:"+presMin+"
Max:"+presMax;

document.getElementById("lightMinMax").innerHTML = "Min:"+lightMin+"
Max:"+lightMax;

// --- GRAPH ---

let time = new Date().toLocaleTimeString();
labels.push(time);
tempData.push(temperature);

```

```

humData.push(humidity);

if(labels.length > 10) {
    labels.shift();
    tempData.shift();
    humData.shift();
}

chart.update();

// --- HISTORY ---
let entry = `<div class="entry">
    [${time}] T:${temperature}°C | H:${humidity}% | P:${pressure}hPa | L:${light}lux
</div>`;

document.getElementById("historyBox").innerHTML =
    entry + document.getElementById("historyBox").innerHTML;

// --- ALERT ---
if(temperature > 32) showAlert("🔥 High Temperature!", "danger");
else if(humidity > 70) showAlert("💧 High Humidity!", "warning");
else if(light < 100) showAlert("🌑 Low Light!", "warning");
else if(pressure < 995) showAlert("⚠️ Low Pressure!", "warning");
else showAlert("✅ Normal Conditions", "safe");
}
</script>

</body>
</html>

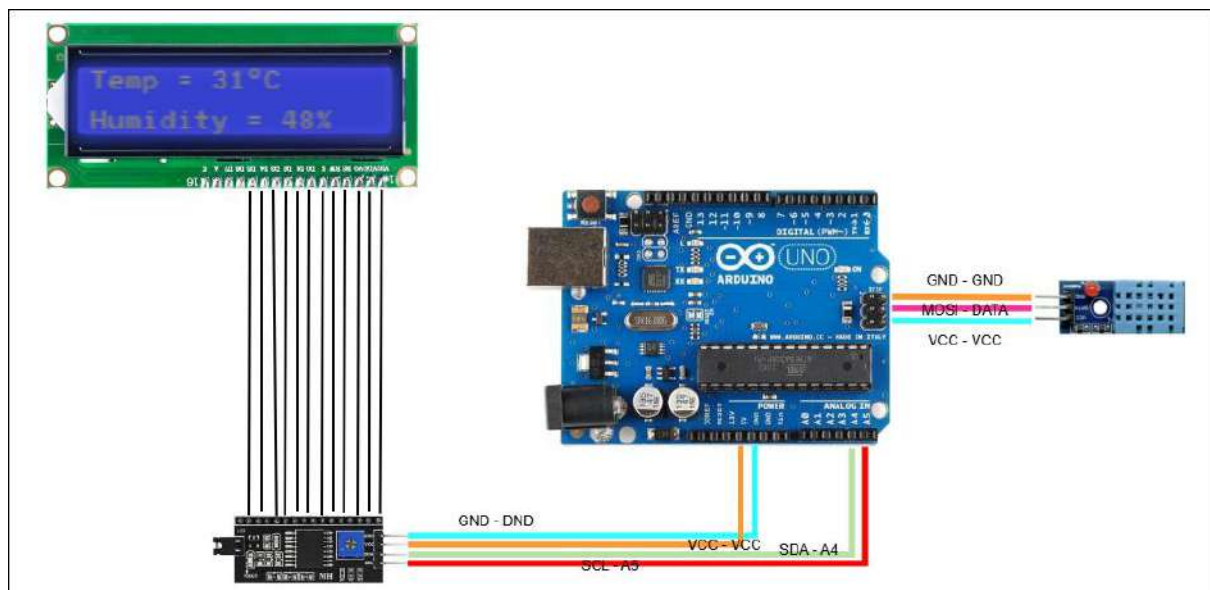
```

Components and Cost Estimation

Item	Description / Notes	Approx. Cost (INR)
Arduino Uno	Main controller board (original or compatible)	₹400
16×2 LCD Display	Used to display temperature and humidity	₹200
I2C Module	Reduces wiring for LCD (SDA & SCL only)	₹100
DHT11 Sensor	Measures temperature and humidity	₹150
Jumper Wires	For circuit connections	₹100
Breadboard	For prototyping connections	₹100
Power Supply / USB Cable	Powering the Arduino	₹100
Item	Description / Notes	Approx. Cost (INR)

Total: 1250

Circuit Diagram



Hardware Setup

